
status: implemented date: 2026-05-18 implemented: 2026-05-18 methodology: [MADR] decision-makers: [Henrik Pettersen] tags: [security, file-permissions, session-state] related: [0082, 0162, 0186] audience: ai-executor

ADR-0188: Session-state file mode — should ~15 raw `writeFileSync` callsites adopt 0600?

Context and Problem Statement

ADR-0186's "DB-write file-mode audit" (originally ADR-0162 follow-up #2) verified that **credential / sensitive** write sites already discipline to 0600:

- `writeFileRestricted` (in `v3/@claude-flow/cli/src/fs-secure.ts`) is the canonical 0600 writer.
- Used at `mcp-tools/terminal-tools.ts:76` (terminal sessions).
- Used at `mcp-tools/session-tools.ts:186` (session vault).
- `memory-router.ts:830` calls `fs.chmodSync(databasePath, 0o600)` on the `memory.db` file.

The audit also found ~15 raw `fs.writeFileSync` callsites that write **session-state JSON without explicit mode** (will inherit the process umask, typically 0644 on macOS / 0644 on Linux):

File	Lines	Content written
<code>v3/@claude- flow/cli/src/init/helpers- generator.ts</code>	99, 113, 131, 167, 180, 1154, 1166, 1181, others	SESSION_FILE JSON (user-prompt session state)
<code>v3/@claude- flow/cli/src/commands/hive-mind- session.ts</code>	413	compressed session blob to tmpPath
<code>v3/@claude- flow/cli/src/commands/session.ts</code>	590	session content utf-8

These files contain **session state**, which can include user prompts, AI responses, and intermediate tool inputs/outputs. They are not credentials in the same equivalence class as bearer tokens or keys, but they are non-public material.

Question: should these callsites adopt 0600 (treating session content as sensitive) via `writeFileRestricted`, or stay at 0644 (the inherited umask default, treating session JSON as project-local ephemeral state)?

Decision Drivers

- **Threat model.** A multi-tenant machine where the session JSON is in a project directory readable by other local users would expose user prompts. On single-user dev machines this is moot. CI hosts vary.
- **Defense-in-depth philosophy.** ADR-0082 disallows silent failure; the spirit of "fail loud" extends to "don't leak silently either." Defaulting to 0600 for content that *might* contain sensitive material is conservative.
- **Refactor surface area.** ~15 callsites across 3 files. Mechanical change but not zero — every conversion needs the import of `writeFileRestricted`, possibly options-block conversion if the existing call uses the `encoding` argument.
- **API ergonomics.** `writeFileRestricted` already supports the signatures these callsites use: plaintext, with-encoding, with-options-object. Conversion is grep-and-replace plus import.

- **No outstanding incident report.** No user has complained about session JSON exposure; this is preventive hardening.

Considered Options

1. **Convert all to `writeFileRestricted`** . Mechanical refactor of ~15 callsites; ship in one commit. Treats session JSON as sensitive at-rest. Highest defense; small refactor cost.
2. **Keep 0644 as design intent.** Document that session JSON is ephemeral project-local state, not sensitive. No code change. Lowest defense; zero cost.
3. **Convert only `helpers-generator.ts` session writes (the bulk).** 12+ `SESSION_FILE` callsites → `writeFileRestricted` . Leave `hive-mind-session.ts` and `session.ts` at 0644 (these write transient/compressed state that's less likely to contain user prompts).
4. **Add a `writeFileRestricted` lint rule** that flags any new `fs.writeFileSync` callsite without mode. Doesn't fix the existing ~15; prevents new ones.

Decision Outcome

Option 2 — keep 0644 as design intent. Session-state JSON and compressed session archives are project-local ephemeral material: session snapshots, agent prompts, tool inputs/outputs scoped to the working project directory. They are not credential equivalent (no bearer tokens, no API keys), they're already inside a directory the user controls, and on the single-user dev-machine deployment mode (per project's `user_machine` memory) there is no second user to protect against. The threat model that motivates 0600 — multi-tenant host with co-located unprivileged readers — is not the project's supported posture.

Implementation: one doc note in `fs-secure.ts` recording the boundary so a future security audit finds the rationale next to the code, not only in this ADR. Fork commit `5be65f3bf` on `forks/ruflo/main` (2026-05-18).

The credential-equivalence-class writes already use `writeFileRestricted` :

- `mcp-tools/terminal-tools.ts` — terminal session vault
- `mcp-tools/session-tools.ts` — MCP session vault
- `memory-router.ts:830` — `chmod 0o600` on `memory.db`

Everything else stays at the inherited umask.

History — Option 1 was attempted and reverted

A first pass implemented Option 1 (fork commit `5228ffc445` on 2026-05-18) converting two real TS-level callsites to `writeFileRestricted` and surfacing that the 12+ "callsites" in `init/helpers-generator.ts` were actually inside template literals emitted as generated CommonJS — making conversion structurally unsound for that file. After the surface area was clarified, Option 2 was chosen instead: commit `5228ffc445` was reverted via `634747deb`, and the doc-note landed via `5be65f3bf`. The helpers-generator gap dissolves under Option 2 because there is nothing to convert there either.

Consequences

If Option 1 (convert all):

- Single PR, ~15 callsite edits + 3 file imports.
- Lower regression risk than expected: `writeFileRestricted` already preserves the existing call signatures.
- All session-state writes uniformly at 0600 — defense-in-depth consistent with credential vault writes.

If Option 2 (keep 0644):

- No code change; one-paragraph doc note in `fs-secure.ts` explaining why session JSON is NOT a `writeFileRestricted` consumer.
- Risk: future security audit re-encounters the same finding without the rationale captured in code.

If Option 3 (partial conversion):

- Compromise position; introduces a soft boundary between "session state" callsites that need 0600 and ones that don't. The boundary has to be re-justified every time someone adds a new write site.

If Option 4 (lint rule):

- Future-proofs without fixing the existing surface.
- Adds a tooling dependency (eslint plugin or custom rule); maintenance cost.

This ADR closes when Option 1/2/3/4 is chosen.